Data Requirement Analysis and System Study

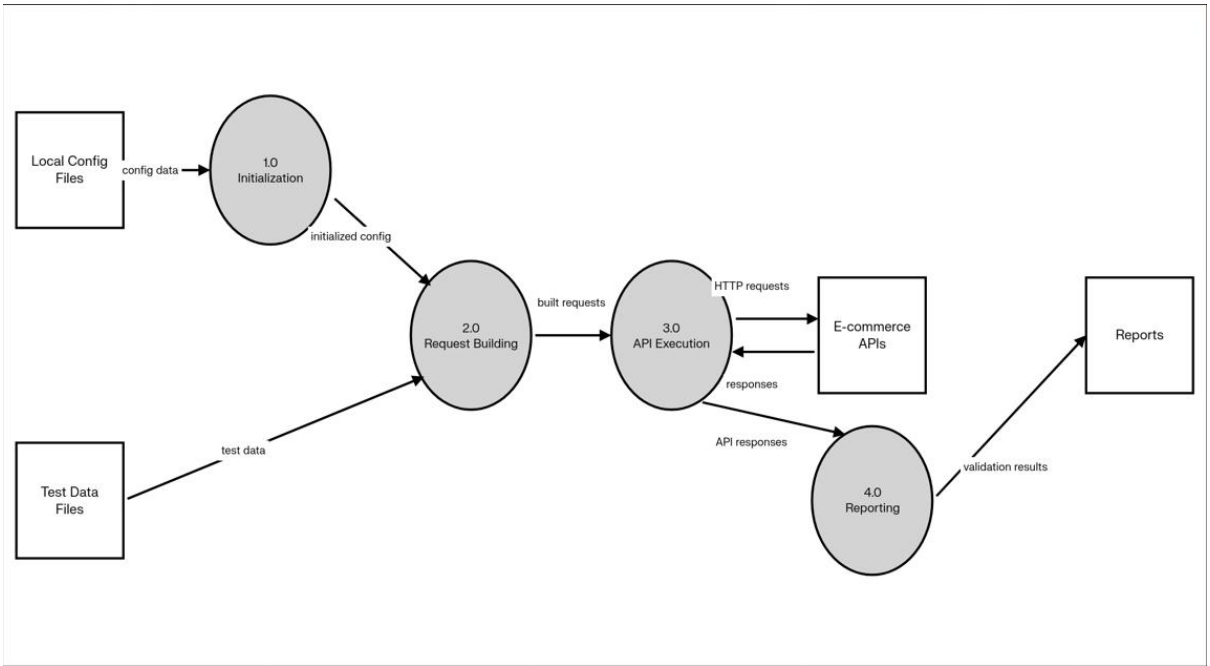
1. System Diagrams

Data Flow Diagram (DFD) - Level 1

The Level 1 DFD shows data flowing from local config and test files through four key processes—Initialisation (1.0 loading karate-config.js), Request Building (2.0 from JSON/CSV), API Execution (3.0 to e-commerce endpoints), and Reporting (4.0 for HTML/JUnit outputs)—to external APIs.

The DFD below illustrates how data moves from local configuration and test data files through the Karate engine to the external e-commerce APIs.

- **Process 1.0 (Initialisation):** Karate loads karate-config.js containing base URLs and authentication parameters.
- **Process 2.0 (Request Building):** The engine reads local JSON/CSV files to construct HTTP requests for specific e-commerce flows.
- **Process 3.0 (API Execution):** Requests are sent to **testJSON** or **TestStore** endpoints.
- **Process 4.0 (Reporting):** API responses are validated against expected results, and the output is written to HTML and JUnit XML reports.



Entity Relationship Diagram (ERD)

The ERD models core entities like User & Auth (with userId, token), Product (id, price), Shopping Cart (linking userId to products with quantity/total), and Reporting (status, response time), with relationships such as one-to-many from Cart to Products.

Copyright © 2026 - Proprietary - OSS Java Hub. All Rights Reserved.

The ERD below identifies the logical data entities managed by the framework to simulate e-commerce operations.

- **User & Auth Entity:** Attributes include userId, username, password, token, and sessionId.
- **Product Entity:** Attributes include id, title, price, category, and stock.
- **Shopping Cart Entity:** The central entity linking a userId to a products array containing productId, quantity, and total.
- **Reporting Entity:** Captures execution data such as Scenario Name, HTTP Status, Response Time, and Error Messages.

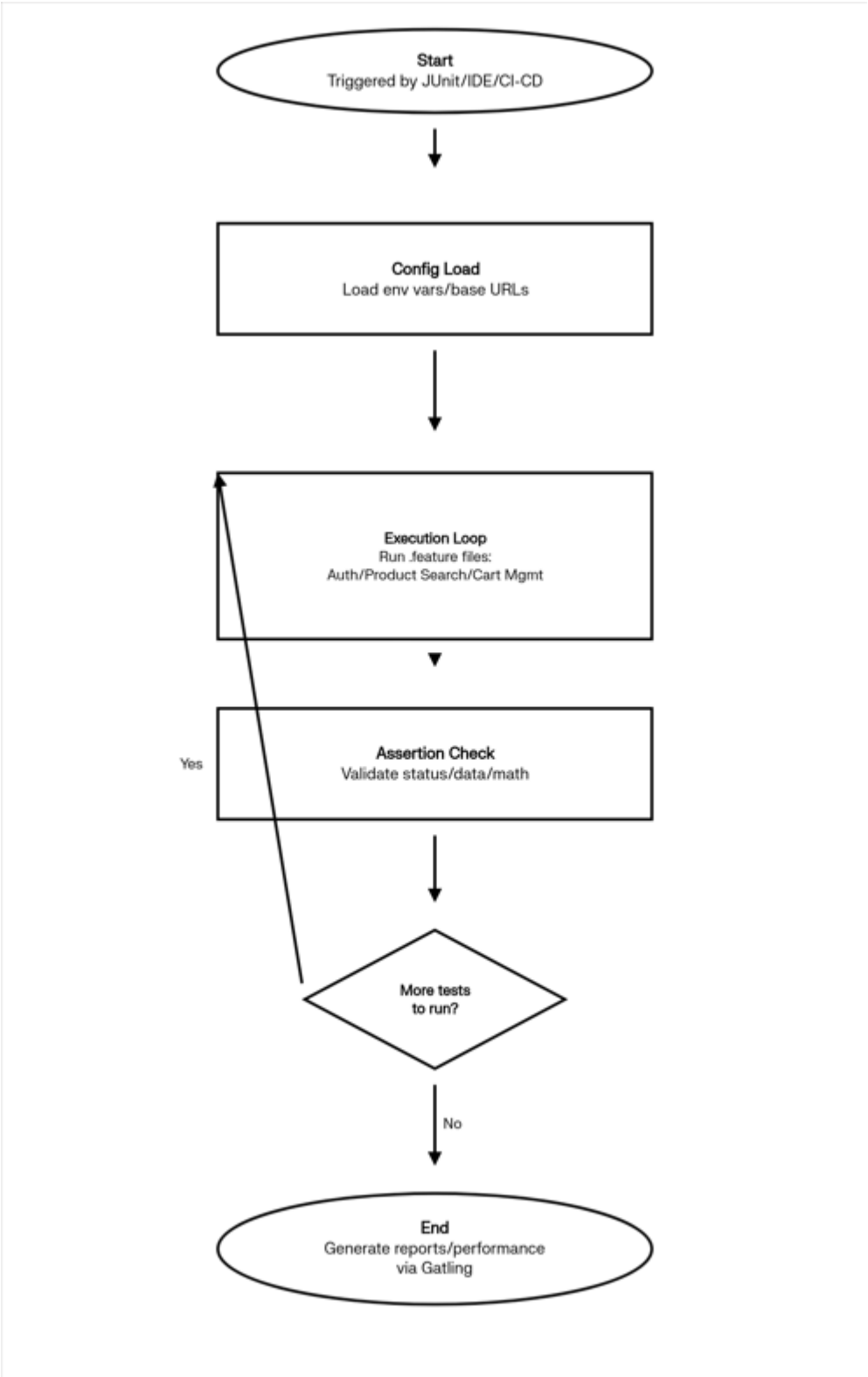
System Flowchart

This flowchart sequences execution: starting via JUnit/CI, loading config, looping through feature files (Auth, Search, Cart), checking assertions (status, math), and ending with Gatling reports

The below flow chart outlines the operational sequence of the Karate automation suite execution from start to finish.

1. **Start:** Triggered via JUnit runner in the IDE or CI/CD pipelines (Jenkins/GitHub Actions).
2. **Config Load:** Load global environment variables and base URLs.
3. **Execution Loop:** Run modular .feature files for Authentication, Product Search, and Cart Management.
4. **Assertion Check:** Validate status codes, data structures, and mathematical consistency (e.g., total price).
5. **End:** Generate final reports and performance metrics via Gatling integration.

Copyright © 2026 - Proprietary - OSS Java Hub. All Rights Reserved.



Copyright © 2026 - Proprietary - OSS Java Hub. All Rights Reserved.

2. Data System Requirements

2.1 Functional Data Requirements

- **Endpoint Interaction:** The framework shall send requests to cart and product endpoints and validate response fields and status codes.
- **Mathematical Validation:** The system shall verify that totals, discounted totals, and quantities match the product list returned by the API.
- **Data-Driven Tests:** The framework must support varying userId, productId, and quantity values using Scenario Outline tables.
- **State Consistency:** The system must ensure data remains consistent across retrieval operations (e.g., POST responses must match subsequent GET responses).

2.2 Non-Functional Data Requirements

- **Data Integrity:** Aggregate values like totalQuantity must remain consistent throughout a test scenario, even when multiple endpoints are called.
- **Security & Privacy:** Real customer data is not used; the framework relies on dummy data and artificial credentials to avoid privacy concerns.
- **Performance Metrics:** Response times, status codes, and error rates must be captured during load tests using Gatling.
- **Scalability:** The framework must support adding new modules (e.g., orders or payments) without redesigning existing data structures.

2.3 Data Storage & Structure

- **JSON Documents:** Request bodies and expected fragments are stored as external JSON files or inline JSON within feature files.
- **Tabular Data:** Parameter sets for bulk testing are represented in CSV, Excel, or Gherkin tables.
- **Version Control:** All data artifacts (test code, JSON files, config) are stored in a Git repository for change tracking.

© Copyright © 2026 - Proprietary - OSS Java Hub. All Rights Reserved.